Solution: Sliding Window Maximum

Problem Statement

You are given an array of integers `nums`, and integer `k`, size of the sliding window which moves from the very `left` to the very `right` in the array. In each window, you can have only `k` numbers, and the window moves one position right by each time.

Return the array containing the `maximum element` of each sliding window.

Examples

Example 1:

- **Input**: nums = [2, 1, 5, 1, 3, 2], k = 3
- **Expected Output**: [5, 5, 5, 3]
- **Justification**: Here, the window of size 3 moves across the array. The maximum values in each window are 5 (from [2, 1, 5]), 5 (from [1, 5, 1]), 5 (from [5, 1, 3]), and 3 (from [1, 3, 2]).

Example 2:

- **Input**: nums = [4, -2, -3, 4, 1], k = 2
- **Expected Output**: [4, -2, 4, 4]
- **Justification**: The maximum values in each window of size 2 are 4 (from [4, -2]), -2 (from [-2, -3]), 4 (from [-3, 4]), and 4 (from [4, 1]).

Example 3:

- **Input**: nums = [9, 7, 2, 4, 6], k = 4
- **Expected Output**: [9, 7]
- **Justification**: In this case, there are only two windows of size 4. The maximum value in the first window is 9 (from [9, 7, 2, 4]), and in the second window is 7 (from [7, 2, 4, 6]).

Solution

To solve this problem, we'll use a deque (double-ended queue) to maintain the indices of the elements in the current window, ensuring the deque always has the maximum element's index at the front. This approach works because it allows us to efficiently remove elements that are no longer in the window (by checking if they are out of the current window's range) and ensure the first element in the deque is always the maximum of the current window. We'll iterate through the array, adding each element's index to the deque while maintaining these properties.

This method is effective because it combines the efficiency of a queue for managing the window's sliding mechanism with the ability to quickly update and retrieve the maximum value. By only storing relevant indices and removing those that are no longer in the window or are not potential maximums, we significantly reduce the time complexity compared to brute force methods that check every window's maximum separately.

Step-by-step Algorithm

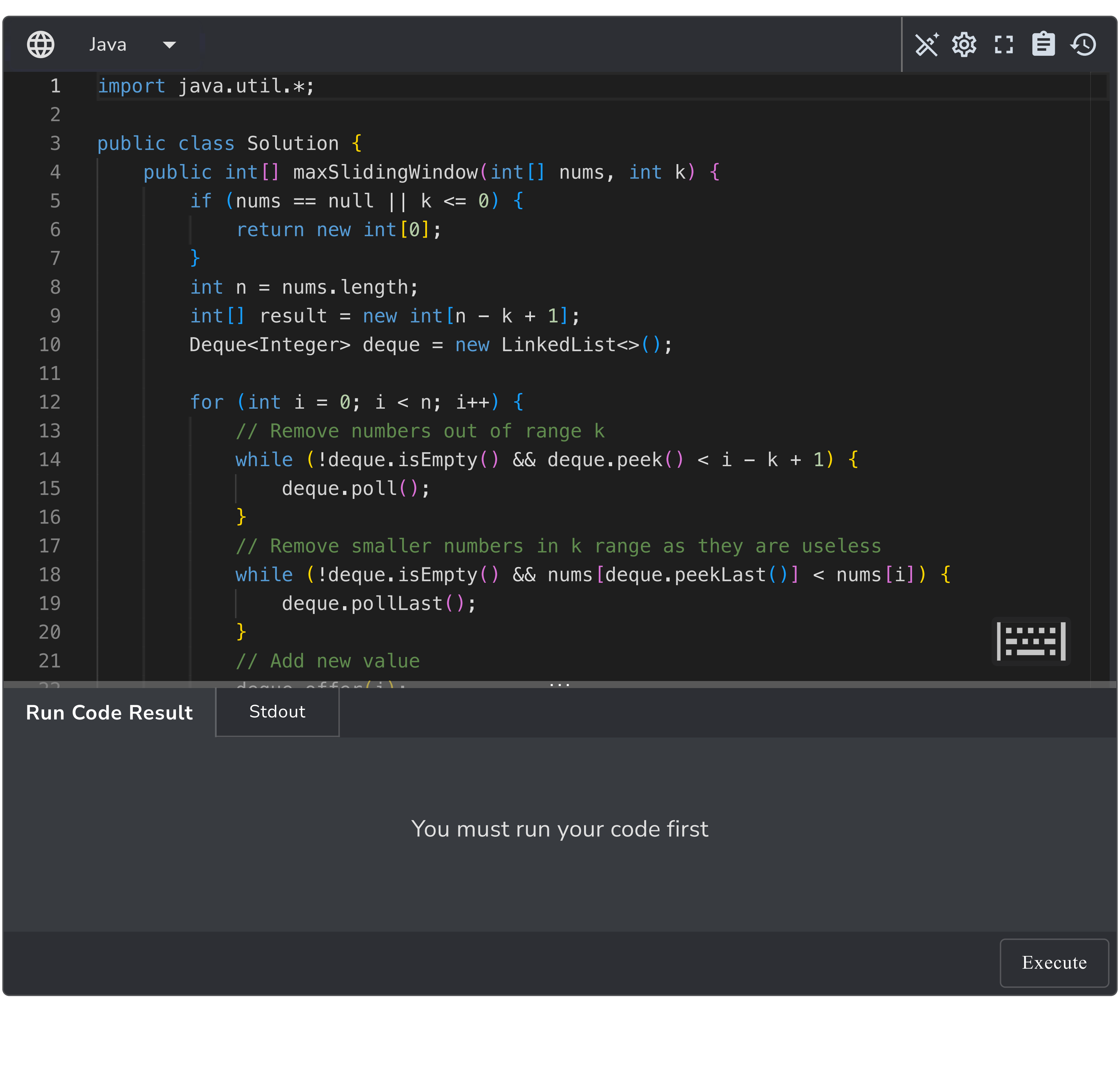
- Initialize an empty deque `D` to store indices of array elements.
- Iterate through each element of the array `nums`:
  - If `D` is not empty and the front element of `D` is out of the current window's range (`i - k`), pop it from the front.
  - While `D` is not empty and the current element is greater than the element at the back of `D`, pop elements from the back. This ensures that the first element in `D` is always the maximum of the current window.
  - Add the current index `i` to the back of `D`.
  - If `i` is greater than or equal to `k - 1`, append the front element of `D` to the result array, as the window has now reached its full size.
- Return the result array containing the maximums.

Algorithm Walkthrough

Let's consider the input: nums = [2, 1, 5, 1, 3, 2], k = 3

1. **Initialization**:
   - The deque `D` is initialized as empty.
   - An empty result array is initialized to store the maximum of each window.
2. **Iteration 1** (**i = 0, nums[i] = 2**):
   - Since `D` is empty, add index `0` to `D`.
   - `D` now contains `[0]`.
   - The window has not yet reached its full size (`k = 3`), so we don't add anything to the result array.
3. **Iteration 2** (**i = 1, nums[i] = 1**):
   - No indices are removed from `D` since all indices in `D` are within the current window's range.
   - `nums[i]` is less than `nums[D.back()]` (1 < 2), so just add index `1` to `D`.
   - `D` now contains `[0, 1]`.
   - The window has not reached its full size, so no action on the result array.
4. **Iteration 3** (**i = 2, nums[i] = 5**):
   - No indices are removed from `D` for being out of range.
   - `nums[i]` is greater than `nums[D.back()]`, so remove indices from `D` while `nums[i]` is greater than `nums[D.back()]`. Remove `0` and `1` from `D`.
   - Add index `2` to `D` since `nums[2]` is now the greatest so far.
   - `D` now contains `[2]`.
   - The window has reached its full size. Add `nums[D.front()]` (which is `5`) to the result array.
5. **Iteration 4** (**i = 3, nums[i] = 1**):
   - No indices are removed from `D` for being out of range.
   - `nums[i]` is less than `nums[D.back()]`, so just add index `3` to `D`.
   - `D` now contains `[2, 3]`.
   - Add `nums[D.front()]` (which is still `5`) to the result array since the window is full.
6. **Iteration 5** (**i = 4, nums[i] = 3**):
   - No indices are removed from `D` for being out of range.
   - `nums[i]` is greater than `nums[D.back()]` (the index at `D.back()` is `3`, and `nums[3]` is `1`), so remove index `3` from `D`.
   - Add index `4` to `D`.
   - `D` now contains `[2, 4]`.
   - Add `nums[D.front()]` (which is `5`) to the result array.
7. **Iteration 6** (**i = 5, nums[i] = 2**):
   - Remove index `2` from `D` since it's out of the current window's range (`i - k + 1 = 3`).
   - `nums[i]` is less than `nums[D.back()]`, so just add index `5` to `D`.
   - `D` now contains `[4, 5]`.
   - Add `nums[D.front()]` (which is `3`) to the result array.
8. **Final Result**:
   - The result array, which contains the maximum of each window, is `[5, 5, 5, 3]`.

Code



```java
import java.util.*;

public class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || k <= 0) {
            return new int[0];
        }
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> deque = new LinkedList<>();

        for (int i = 0; i < n; i++) {
            // Remove numbers out of range k
            while (!deque.isEmpty() && deque.peek() < i - k + 1) {
                deque.poll();
            }
            // Remove smaller numbers in k range as they are useless
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }
            // Add new value
```

You must run your code first

Complexity Analysis

Time Complexity

- $O(N)$: Each element in the array is processed exactly twice - once when it's added to the deque and once when it's removed. This leads to a linear time complexity because the operations of adding to and removing from the deque (or its equivalent data structure) are O(1) operations.

Space Complexity

- $O(N)$: In the worst-case scenario, the space complexity is $O(N)$ due to the storage requirements of the result array, which contains at most N - k + 1 elements. The deque (or similar structure) itself will have at most k elements because it only stores elements from the current window. However, since k is a constant that does not scale with the input size N, the dominant term for space complexity is due to the output array, making it $O(N)$.